

Least-to-Most Prompting y Descomposición

Módulo 6 · Ingeniería de Prompts

Descomposición progresiva, prompt chaining, MapReduce y diseño de pipelines LLM como arquitectura de software.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Least-to-Most Prompting y Descomposición de Tareas

Hay una clase de problemas donde CoT estándar falla sistemáticamente: los problemas compositivos que requieren combinar múltiples habilidades o resolver subproblemas que dependen unos de otros en forma de árbol. Un contrato de 50 páginas con 30 cláusulas que deben analizarse individualmente y luego sintetizarse. Un análisis de mercado que requiere datos de 5 fuentes distintas, síntesis sectorial, y valoración estratégica. Una migración de sistema con 15 módulos interdependientes. Estos problemas colapsan bajo CoT simple porque exceden la capacidad de razonamiento en un único contexto.

Least-to-Most prompting (Zhou et al., 2022) introduce una solución elegante: descomponer el problema en subproblemas ordenados de menos difícil a más difícil, resolver cada subproblema usando las soluciones de los anteriores como contexto, y construir progresivamente hacia la solución completa. Es la forma de abordar los problemas más complejos de manera sistemática y reproducible.

El principio de descomposición progresiva

Least-to-Most: de lo simple a lo complejo

Least-to-Most tiene dos fases. La fase de reducción descompone el problema original en una secuencia de subproblemas más simples, ordenados para que cada uno pueda resolverse con las soluciones de los anteriores. La fase de resolución resuelve los subproblemas en orden, usando las soluciones anteriores como parte del contexto para resolver el siguiente. El modelo nunca necesita razonar sobre el problema completo de una vez; siempre está resolviendo el subproblema más simple que queda pendiente.

Prompt chaining: encadenar modelos para tareas complejas

Prompt chaining es una implementación práctica de la descomposición: en lugar de un único prompt que intenta hacer todo, se construye una cadena de prompts especializados donde el output de cada uno es el input del siguiente. Esto permite: especializar cada prompt para su subtarea específica, usar diferentes modelos o configuraciones para cada etapa, auditar el output de cada etapa por separado, y reintentar solo la etapa que falla sin reejecutar todo el pipeline.

Descomposición funcional vs descomposición secuencial

La descomposición funcional divide el problema por tipo de operación: primero extrae todos los datos, luego analiza cada elemento, luego sintetiza. La descomposición secuencial divide el problema por orden temporal o lógico: primero resuelve los subproblemas más básicos que no dependen de nada, luego los que dependen de los anteriores, hasta llegar al problema principal. La elección depende de la naturaleza del problema: la funcional es mejor para pipelines de datos; la secuencial es mejor para problemas de razonamiento jerárquico.

Implementación de pipelines de descomposición

Un pipeline de descomposición bien diseñado tiene: una etapa de análisis que identifica los subproblemas y su orden, etapas de resolución especializadas para cada subproblema, mecanismos de verificación entre etapas para detectar errores antes de propagarlos, y una etapa de síntesis que combina todas las soluciones parciales en la solución final.

En LangChain y LlamaIndex, las cadenas (chains) son la implementación estándar del prompt chaining. LangChain Expression Language (LCEL) permite definir pipelines de forma declarativa: `extraction_chain | analysis_chain | synthesis_chain`. Cada link de la cadena es un Runnable que recibe el output del anterior. El estado se puede mantener entre etapas con memory o con estructuras de datos explícitas pasadas como part del contexto.

MapReduce para problemas de gran escala

Para problemas con muchos elementos del mismo tipo (resumir 50 documentos, analizar 100 transacciones, revisar 30 cláusulas contractuales), el patrón MapReduce es la implementación más eficiente: en la fase Map, aplicar el mismo prompt en paralelo a cada elemento independiente; en la fase Reduce, sintetizar todos los resultados individuales en una conclusión global. Este patrón escala horizontalmente y permite procesar volúmenes que exceden la ventana de contexto de cualquier modelo.

SECCIÓN 4 · EJEMPLOS REALES

- Revisión de contratos con MapReduce: etapa 1 (Map) → analizar cada cláusula independientemente para identificar tipo, obligaciones y riesgos. Etapa 2 (Reduce) → sintetizar todos los análisis de cláusulas en un resumen ejecutivo del contrato con los riesgos más importantes clasificados por severidad. La descomposición permite procesar contratos de cualquier longitud.
- Análisis de due diligence empresarial: descomposición en subproblemas independientes (análisis financiero, análisis legal, análisis de mercado, análisis de equipo) resueltos en paralelo, seguido de una síntesis que integra todos los análisis en una valoración global con recomendación de inversión.
- Generación de informes de mercado: cadena de prompts especializados: extracción de datos de múltiples fuentes → normalización y limpieza de datos → análisis de tendencias por segmento → comparación con competidores → síntesis y recomendaciones → redacción del informe en el formato corporativo estándar. Cada etapa tiene su propio prompt optimizado para esa tarea específica.
- Onboarding de nuevos clientes en una aseguradora: pipeline de verificación con Least-to-Most: verificar identidad → calcular perfil de riesgo básico → identificar productos elegibles → calcular primas para cada producto → generar propuesta personalizada. Cada paso usa el output del anterior como contexto.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Propagación de errores en cascada. Si la etapa 1 produce un error y no se detecta antes de la etapa 2, el error se propaga y amplifica a través del pipeline. Implementa verificaciones de calidad entre etapas para detectar outputs inválidos antes

de que lleguen a la siguiente etapa.

Error 2: Descomposición demasiado granular. Descomponer en demasiados pasos pequeños añade overhead de gestión, aumenta el número de llamadas al API, y puede fragmentar el contexto necesario para el razonamiento. Encuentra el nivel de granularidad donde cada subtarea es manejable pero no trivial.

Error 3: No aprovechar el paralelismo. Si los subproblemas son independientes entre sí, deben resolverse en paralelo para minimizar la latencia total. Una cadena de 10 pasos secuenciales puede tener 10x la latencia de 10 pasos paralelos.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de diseño de un pipeline de descomposición: traza el problema completo en un diagrama de flujo, identifica cuáles son las etapas independientes (paralelizables) y cuáles son dependientes (secuenciales), diseña un prompt especializado para cada etapa con instrucciones precisas del formato de input y output, implementa las verificaciones de calidad entre etapas (schema validation, assertions básicas), y monitoriza las métricas de calidad de cada etapa por separado.

Para el patrón MapReduce en Python con la API de OpenAI: usa `asyncio.gather()` para las llamadas Map en paralelo, collect los resultados en una lista, y pasa esa lista al prompt de reducción. El overhead de latencia del Map en paralelo es similar a una sola llamada; el de Map en serie sería N veces mayor.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M6-T2 (CoT): Least-to-Most es una extensión de CoT que estructura los pasos de razonamiento en una jerarquía.
- M7 (RAG): los sistemas RAG complejos usan descomposición para separar la recuperación, el análisis y la generación en etapas distintas.
- M10 (Agentes): los agentes usan descomposición para planificar la secuencia de acciones necesarias para completar tareas complejas.

SECCIÓN 8 · RESUMEN EJECUTIVO

Least-to-Most prompting descompone problemas complejos en subproblemas ordenados de simple a complejo, resolviendo cada uno con los resultados anteriores como contexto. Prompt chaining encadena prompts especializados en pipelines donde el output de cada etapa es el input de la siguiente. MapReduce procesa muchos elementos en paralelo (Map) y sintetiza los resultados (Reduce). Las reglas prácticas: detectar errores entre etapas para evitar propagación, paralelizar las etapas independientes, encontrar el nivel de granularidad óptimo. El diseño del pipeline de prompts es una decisión de arquitectura de software con sus propios requisitos de modularidad, testeabilidad y mantenibilidad.